

1. qbp-sim User Guide

qbp-sim is a continuous-time Gillespie simulator for quantum-network routing experiments inspired by quantum backpressure. The package is designed to be used from typed JSON configs and the **qbp-sim** command-line interface. A small Python facade is available for scripts that need programmatic control.

The simulator is not a line-by-line reproduction of the paper's slotted theorem. It is a continuous-time analogue with explicit event traces, replay, and experiment helpers. The LP module is a separate global average-rate benchmark used to generate or compare against simulation configs; it is not the same controller as the stochastic simulator.

2. Installation

Use `uv` for this repository:

```
uv sync --all-groups
uv run qbp-sim --help
```

Build this manual with Typst:

```
typst compile docs/manual.typ docs/qbp-sim.pdf
```

3. Quick Start

Run the built-in four-node example:

```
uv run qbp-sim example --until 50 --seed 0
```

Run from a JSON config:

```
uv run qbp-sim run --config docs/examples/basic_config.json --until 50
```

Save a compact Vortex event trace:

```
uv run qbp-sim run \
  --config docs/examples/basic_config.json \
  --until 50 \
  --trace output/traces/basic.vortex
```

Replay a trace against the built-in example:

```
uv run qbp-sim replay --trace output/traces/basic.vortex --until 50
```

Write sampled snapshots and analyze them:

```
uv run qbp-sim run \
  --config docs/examples/basic_config.json \
  --until 50 \
  --snapshots output/snapshots/basic.jsonl.zst
uv run qbp-sim analyze \
  --snapshots output/snapshots/basic.jsonl.zst \
  --plot-metric service_ratio \
  --plot-out output/plots/basic_service_ratio.png
```

4. Simulation Configs

A simulation config describes an undirected network with symmetric matrices. The row and column index is the node id.

- `generation_rates[x][y]`: Bell-pair generation hazard on edge (x,y) . Positive entries imply physical generation edges.
- `consumption_rates[x][y]`: demand arrival rate for service on pair (x,y) .
- `swap_rates[i]`: per-node swap opportunity rate.
- `capacity_headroom`: multiplier applied to controllable generation, swap, and service opportunity rates. Demand arrivals are not scaled.
- `virtual_swap_policy`: swap-selection policy. The default `bp` is full-information backpressure.

Minimal example:

```
{
  "generation_rates": [
    [0.0, 1.0, 1.0, 0.0],
    [1.0, 0.0, 0.0, 1.0],
    [1.0, 0.0, 0.0, 1.0],
    [0.0, 1.0, 1.0, 0.0]
  ],
  "consumption_rates": [
    [0.0, 0.0, 0.0, 2.0],
    [0.0, 0.0, 0.0, 0.0],
    [0.0, 0.0, 0.0, 0.0],
    [2.0, 0.0, 0.0, 0.0]
  ],
  "swap_rates": [0.0, 1.0, 1.0, 0.0],
  "capacity_headroom": 1.01,
  "virtual_swap_policy": {"mode": "bp"}
}
```

Supported policy modes:

- **bp**: full-information backpressure. Each node scans all swap candidates and chooses the largest positive virtual pressure.
- **limited_info_bp**: limited-information backpressure. Each node samples k candidates, remembers the best memory candidates, and exposes only the best positive remembered candidate.
- **max_min**: path-oblivious inventory balancing baseline. It uses physical inventory Q , not backpressure scarcity α .
- **limited_info_max_min**: limited-information max-min. Each node applies the same k -query, memory-candidate restriction to max-min swap selection.

5. Running Simulations

`qbp-sim run` is the main command for user-provided configs:

```
uv run qbp-sim run --config CONFIG.json --until 100 --seed 1
```

Common options:

- `--max-events 0`: no event cap. Positive values stop early after that many sampled events.
- `--sample-every N`: record aggregate snapshots every N applied events.
- `--trace OUTFILE`: write every event. Use `.vortex`, `.parquet`, or `.jsonl.zst`.
- `--trace-float-precision float32`: precision for columnar trace floats.
- `--trace-time-mode none`: omit time/rate fields for smaller event-order traces.
- `--snapshots OUTFILE`: write sampled aggregate snapshots.
- `--instant-service-fulfillment`: deterministic immediate service realization when new inventory satisfies pending service on the same edge.
- `--instant-swap-fulfillment`: deterministic immediate realization for feasible pending swaps.

Traces are for replay and detailed later analysis. Snapshots are sampled aggregates for plotting and quick summaries.

6. Experiment Matrices

Use an experiment matrix when you want a small sweep without writing Python:

```
uv run qbp-sim matrix \
  --config docs/examples/matrix_config.json \
  --output-dir output/matrix_demo
```

Inspect the cases without running them:

```
uv run qbp-sim matrix \
  --config docs/examples/matrix_config.json \
  --output-dir output/matrix_demo \
  --dry-run
```

Each completed case writes:

- `lp_solution.json`: LP benchmark output used to derive the config.
- `simulation_config.json`: exact simulator input after headroom and policy settings.
- `events.vortex`: event trace.
- `run_metadata.json`: seed, horizon, paths, settings, and result counters.

The matrix output directory also contains `summary.csv` with one row per case.

7. Interpreting Outputs

The primary service metric is:

```
service_ratio = services_completed / demand_arrivals
```

Important counters:

- `total_backlog`: aggregate pending virtual or physical work.
- `total_inventory`: Bell pairs currently stored in Q.
- `total_scarcity`: aggregate backpressure scarcity alpha.
- `pair_generations`: direct generated Bell pairs.
- `swaps_completed`: physical swaps completed.

If a trace is written with `--trace-time-mode none`, replay preserves event order and state transitions but cannot reconstruct the real simulation clock unless you provide an external final time.

8. Output Examples

A typical run without file outputs prints a summary like:

```
QBP Gillespie simulation
time=50.000000
events=292
backlog=14
inventory=9
scarcity=5
demand_arrivals=87
pair_generations=101
virtual_service_requests=80
virtual_swap_requests=24
services_completed=73
swaps_completed=22
```

A `run_metadata.json` file records reproducibility data and final counters:

```
{
  "schema_version": 1,
  "command": "matrix",
  "n_nodes": 3,
  "seed": 3,
  "until_time": 2.0,
  "trace_time_mode": "none",
  "simulation_config_path": ".../simulation_config.json",
  "trace_path": ".../events.vortex",
  "result": {
    "final_time": 2.0,
    "events_processed": 8,
    "demand_arrivals": 1,
    "services_completed": 1,
    "service_ratio": 1.0
  }
}
```

A matrix `summary.csv` contains one row per resolved case. Important columns include:

slug, topology, n_nodes, capacity_headroom, policy_label, seed, final_time,
events_processed, demand_arrivals, services_completed, service_ratio,
trace_path, metadata_path, simulation_config_path

9. Plots

All built-in plots are Altair/Vega-Lite charts and can be written as .png, .svg, .html, or .json depending on the output filename.

9.1. Snapshot Metric Plot

Use this when you have a snapshot file and want one metric over time:

```
uv run qbp-sim run \  
  --config docs/examples/basic_config.json \  
  --until 50 \  
  --sample-every 25 \  
  --snapshots output/snapshots/basic.jsonl.zst  
  
uv run qbp-sim analyze \  
  --snapshots output/snapshots/basic.jsonl.zst \  
  --plot-metric service_ratio \  
  --plot-out output/plots/basic_service_ratio.png
```

Available snapshot metrics are event_index, time, total_backlog, total_inventory, total_scarcity, demand_arrivals, pair_generations, services_completed, swaps_completed, and service_ratio.

Python equivalent:

```
from qbp_sim.snapshots import SnapshotReader  
from qbp_sim.analysis import plot_snapshot_metric  
  
with SnapshotReader("output/snapshots/basic.jsonl.zst") as reader:  
    snapshots = list(reader)  
  
plot_snapshot_metric(snapshots, "total_backlog", "output/plots/backlog.svg")
```

9.2. Snapshot Metric Series Plot

Use this from Python to compare the same snapshot metric across several runs:

```
from qbp_sim.analysis import plot_snapshot_metric_series  
from qbp_sim.snapshots import SnapshotReader  
  
def load(path):  
    with SnapshotReader(path) as reader:  
        return list(reader)  
  
plot_snapshot_metric_series(  
    [  
        ("bp", load("output/bp/snapshots.jsonl.zst")),  
        ("limited", load("output/limited/snapshots.jsonl.zst")),  
    ],  
    "service_ratio",  
    "output/plots/service_ratio_series.html",  
    title="service ratio comparison",  
)
```

9.3. Cycle Service-Gap Plot

This command solves LP-derived cycle instances, runs backpressure, and plots 1 - service_ratio on log-log axes for each cycle size:

```
uv run qbp-sim cycle-service-ratio \  
  --sizes 4 8 16 \  
  --until 1000 \  
  --plot-out output/plots/cycle_service_ratio_gap.png
```

The plot answers: how quickly does service deficit decay as the topology size changes?

9.4. Headroom Service-Gap Plot

This command compares capacity-headroom multipliers on one LP-derived cycle instance and plots `1 - service_ratio` on log-log axes:

```
uv run qbp-sim headroom-service-ratio \
  --n 16 \
  --headrooms 1.0 1.01 1.05 \
  --until 1000 \
  --plot-out output/plots/headroom_service_ratio_gap.svg
```

The plot answers: how much capacity slack is needed before service deficit decays cleanly?

9.5. Limited-Information Service-Ratio Plot

This command compares full-information backpressure against one or more `limited_info_bp` policies and plots `service_ratio` over log-scaled time:

```
uv run qbp-sim limited-info-service-ratio \
  --n 8 \
  --limited-policies 1:1 2:2 4:4 \
  --until 1000 \
  --plot-start-time 10 \
  --plot-out output/plots/limited_info_service_ratio.png
```

The plot answers: how close is limited-information routing to full-information routing?

9.6. LP Benchmark Ratio Plot

The LP module has an advanced plot helper that compares path-based routing to the LP benchmark on random torus-grid instances. It plots path/LP ratios for total swaps and maximum per-edge generation:

```
from qbp_sim.lp.linear import plot_swaps_and_maxgen_vs_nodes

plot_swaps_and_maxgen_vs_nodes(
    grid_sizes=[2, 3, 4],
    trials=3,
    save_path="output/plots/lp_path_ratios.png",
    seed=0,
)
```

This is an LP benchmark plot, not a stochastic simulator plot.

10. Public Python API

The package root exposes the consumer API. Treat it as the stable Python interface for normal scripts.

10.1. Public Types

- `SimulationInputConfig`: Pydantic model for JSON-facing simulator configs.
- `VirtualSwapPolicyConfig`: Pydantic model for policy settings inside a simulation config.
- `ExperimentMatrixConfig`: Pydantic model for Cartesian-product experiment sweeps.
- `ExperimentPolicyConfig`: one policy entry in an experiment matrix.
- `ExperimentMatrixCase`: one resolved matrix case.
- `RunOptions`: dataclass of runtime options for `run_simulation`.
- `RunOutput`: dataclass containing the `result`, optional trace path, optional snapshot path, and convenience `service_ratio`.
- `VirtualSwapPolicyMode`: enum values `bp`, `limited_info_bp`, `max_min`, `limited_info_max_min`.
- `TopologyName`: enum values `cycle`, `chain`, `grid`.
- `TraceFloatPrecision`: enum values `float16`, `float32`, `float64`.
- `TraceTimeMode`: enum values `full`, `none`.

10.2. Public Functions

- `load_simulation_config(path)`: load and validate a simulation JSON file.

- `load_experiment_matrix_config(path)`: load and validate a matrix JSON file.
- `build_four_node_example_config()`: return the documented four-node example as `SimulationInputConfig`.
- `run_simulation(config_or_path, options=None)`: run a simulation from a config object or JSON path.
- `replay_trace(trace_path, config=None, final_time=None, sample_every=500, snapshots_path=None)`: replay an event trace.

10.3. Basic Programmatic Run

```
from qbp_sim import RunOptions, build_four_node_example_config, run_simulation
```

```
config = build_four_node_example_config()
output = run_simulation(
    config,
    RunOptions(
        until_time=50.0,
        seed=0,
        trace_path="output/traces/python_example.vortex",
    ),
)
print(output.service_ratio)
```

Example output:

```
0.9736842105263158
```

10.4. Load JSON and Replay Traces

```
from qbp_sim import load_simulation_config, replay_trace, run_simulation
```

```
config = load_simulation_config("docs/examples/basic_config.json")
run = run_simulation(
    config,
    RunOptions(
        until_time=50.0,
        trace_path="output/traces/python_example.vortex",
    ),
)
replayed = replay_trace(
    "output/traces/python_example.vortex",
    config=config,
    final_time=run.result.final_time,
)
assert replayed.result.services_completed == run.result.services_completed
```

10.5. Drive a Small Parameter Sweep

```
from qbp_sim import (
    RunOptions,
    VirtualSwapPolicyMode,
    build_four_node_example_config,
    run_simulation,
)
from qbp_sim.config import VirtualSwapPolicyConfig

base = build_four_node_example_config()

for policy in [VirtualSwapPolicyMode.BP, VirtualSwapPolicyMode.MAX_MIN]:
    config = base.model_copy(
        update={"virtual_swap_policy": VirtualSwapPolicyConfig(mode=policy)}
    )
    output = run_simulation(
        config,
        RunOptions(until_time=100.0, seed=0, sample_every=100),
    )
    print(policy, output.service_ratio)
```

10.6. Expand a Matrix Without Running It

```
from qbp_sim import load_experiment_matrix_config

matrix = load_experiment_matrix_config("docs/examples/matrix_config.json")
for case in matrix.cases():
    print(case.slug, case.policy_label, case.seed)
```

10.7. Trace Analysis With Polars

Use Polars as the canonical table interface for event traces. Vortex files can be opened as a Polars `LazyFrame`:

```
import polars as pl
import vortex as vx

lf = vx.open("output/traces/python_example.vortex").to_polars()
service_ratio = (
    lf.select("event_index", "time", "event_type")
    .with_columns(
        demand_arrivals=(pl.col("event_type") == "demand_arrival").cast(pl.Int64).cum_sum(),
        services_completed=(pl.col("event_type") == "physical_service").cast(pl.Int64).cum_sum(),
    )
    .with_columns(
        service_ratio=pl.when(pl.col("demand_arrivals") > 0)
        .then(pl.col("services_completed") / pl.col("demand_arrivals"))
        .otherwise(0.0)
    )
    .select("event_index", "time", "service_ratio")
    .collect()
)
```

The `examples/` directory has runnable scripts that combine hard-coded typed configs, Vortex traces, Polars metric derivation, and Altair plots. Low-level modules such as `qbp_sim.core`, `qbp_sim.io`, and `qbp_sim.lp` are available for library development and advanced replay analysis. New users should start with the facade, CLI, and Polars trace tables.

11. LP Benchmark

The LP module solves a global average-rate optimization problem. It can produce simulator configs and comparison data, but it is not the same stochastic event process as backpressure. Use LP outputs as aggregate-rate benchmarks or as a convenient way to generate feasible network instances.

The experiment matrix command uses the LP builder internally for `cycle`, `chain`, and `grid` topologies, then runs the Gillespie simulator with the derived rates.

12. Performance and Reproducibility

- Use `pueue` for long simulations, benchmark jobs, and analysis runs that should survive shell sessions.
- Prefer `.vortex` traces for compact columnar event logs.
- Use `float32` trace precision by default; use `float64` when exact time/rate precision matters.
- Use `trace_time_mode: "none"` for smaller traces when event order is enough.
- Record fixed seeds and keep `run_metadata.json` with every run.
- Keep generated runs under `ignored output/`.

13. Building Distributable Artifacts

Build a source distribution and wheel locally:

```
uv build
```

The build writes files like:

```
dist/qbp_sim-0.1.0.tar.gz
dist/qbp_sim-0.1.0-py3-none-any.whl
```

The repository also includes a GitHub Actions workflow that runs tests, compiles this Typst manual, builds the wheel and source distribution, and uploads them as downloadable artifacts. Use the `workflow_dispatch` trigger in GitHub when you want an external consumer to download a fresh package artifact without publishing to PyPI.

14. Glossary

- Q : physical Bell-pair inventory matrix.
- α : backpressure scarcity signal.
- virtual demand backlog: demand accumulated before admission to physical service.
- service backlog, H^R : pending service requests waiting for physical inventory.
- swap deficit, H^μ : pending virtual swaps waiting for physical realization.
- headroom: capacity multiplier applied to generation, swap, and service opportunities, but not to demand arrivals.
- trace: per-event log for replay and detailed analysis.
- snapshot: sampled aggregate state summary.
- service ratio: fulfilled requests divided by requested demand arrivals.